

LABORATORY PROJECT REPORT

COLOR DETECTION AND ANALYSIS

EXPERIMENT 8

Section 1

Group 2

Semester 2 2025/2026

NO	NAME	MATRIC NO
1	AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI	2310663
2	ALYA SYAFINA BINTI ABDUL RAZAK	2218084

DATE OF SUBMISSION: 6 MAY 2026

Abstract

This experiment explores how to detect colors using an AI vision system integrated with a microcontroller. A Gravity HuskyLens AI Vision Camera was connected to an Arduino Uno to identify specific colors and give real-time feedback using an RGB LED (KY-016). The HuskyLens was trained through its built-in color recognition feature to recognize red, green, and blue objects, with each color assigned a unique ID. When a color is detected, the Arduino reads the data and lights up the corresponding color on the RGB LED using Pulse Width Modulation (PWM). The setup highlights how image and video interfacing can be combined with sensor and actuator integration in an embedded system, all without relying on external image processing software. Through this experiment, students gain practical experience in AI-based vision sensing, serial communication, and real-time system response based on visual input, while strengthening their understanding of mechatronics system integration.

Table of Contents

1.0 Introduction

2.0 Materials and Equipment

2.1 Electronic Components

2.2 Equipment and Tools

3.0 Experimental Setup

4.0 Methodology

5.0 Data Collection

6.0 Data Analysis

7.0 Results

8.0 Discussion

9.0 Conclusion

10.0 Recommendations

11.0 References

1.0 Introduction

Image and video interfacing are essential components in modern mechatronic systems, especially in areas such as automation, robotics, and intelligent sensing applications. Conventional color detection systems usually depend on dedicated color sensors, which can be affected by changes in lighting conditions and often require frequent calibration for accurate performance. However, with the development of AI vision technology, devices like the HuskyLens AI Camera offer a more reliable and adaptable solution for visual recognition tasks.

In this experiment, the Gravity HuskyLens AI Vision Camera is used as the input device to identify specific colors through its built-in AI-based color recognition system. The detected color data is then sent to an Arduino Uno via serial communication. To display the output, an RGB LED (KY-016) is used to indicate the detected color instantly in real time.

This experiment focuses on integrating an AI vision sensor with a microcontroller while exploring concepts such as color detection, UART communication, and output control. It demonstrates a straightforward and efficient method of image-based sensing without the need for complex external software or powerful computing systems. The knowledge and skills gained from this experiment can be applied to real-world systems such as automated product sorting, smart traffic control systems, and interactive human–machine interfaces.

2.0 Materials and Equipment

2.1 Electronic Components

- Arduino Uno
- Gravity HuskyLens AI Vision Camera
- KY-016 RGB LED Module
- Jumper Wires
- Breadboard

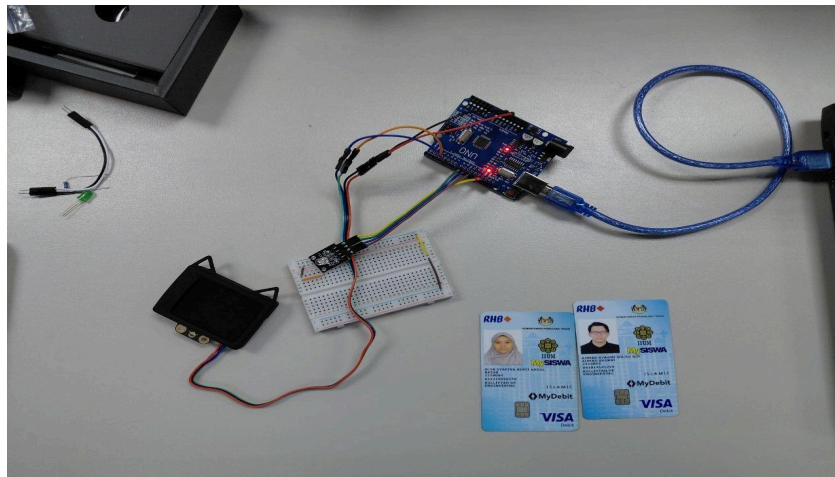
2.2 Equipment and Tools

- Arduino IDE
- USB Cable
- HuskyLens Built-in Display & Buttons

3.0 Experimental Setup

The experimental setup consists of interfacing the HuskyLens AI Camera with an Arduino Uno using UART serial communication and connecting an RGB LED as an output indicator.

1. The HuskyLens camera is connected to the Arduino Uno using SoftwareSerial communication.
2. The HuskyLens VCC and GND pins are connected to the Arduino 5V and GND respectively.
3. The TX and RX pins of the HuskyLens are connected to Arduino digital pins D5 and D6.
4. The KY-016 RGB LED module is connected to the Arduino using PWM-capable pins:
 - Red pin → D9
 - Green pin → D10
 - Blue pin → D11
5. The Arduino Uno is powered through a USB connection to the computer.
6. The HuskyLens is set to Color Recognition Mode, and three colors (red, green, and blue) are trained using the learning button.



4.0 Methodology

The experiment was conducted following a systematic procedure to ensure accurate color detection and reliable hardware response.

4.1 Detailed steps

1. The HuskyLens AI camera was powered on and switched to Color Recognition Mode using the onboard buttons.
2. Three different colored objects (red, green, and blue) were individually trained using the HuskyLens learning button, with each color assigned a unique detection ID.
3. The Arduino Uno was connected to the computer, and the program code was written and uploaded using the Arduino IDE.
4. The program initializes serial communication between the Arduino and HuskyLens using SoftwareSerial.
5. The Arduino continuously requests detection data from the HuskyLens.
6. When a color is detected, the HuskyLens sends the corresponding color ID and position data to the Arduino.
7. Based on the received ID:
 - Red object → Red LED turns ON
 - Green object → Green LED turns ON
 - Blue object → Blue LED turns ON
8. If no object or an untrained color is detected, all RGB LED outputs are turned OFF.
9. The detected color ID and coordinates are displayed on the Serial Monitor for monitoring and debugging.
10. The system is tested under different object positions and lighting conditions to observe detection consistency.

4.1 Programming

Task 1

```
#include "HUSKYLENS.h"
```

```
#include "SoftwareSerial.h"
```

```
HUSKYLENS huskylens;
```

```
SoftwareSerial mySerial(4, 5); // RX, TX
```

```
// Define the LED pin
```

```
const int ledPin = 13;
```

```
void setup() {  
  Serial.begin(115200);  
  mySerial.begin(9600);  
  // Initialize the LED pin as an output  
  pinMode(ledPin, OUTPUT);  
  // Wait for HuskyLens to connect  
  while (!huskylens.begin(mySerial)) {  
    Serial.println(F("Begin failed! Check wiring and Protocol (Serial 9600)"));  
    delay(500);  
  }  
  Serial.println("HuskyLens Ready!");  
}  
  
void loop() {  
  if (!huskylens.request()) {  
    // If request fails or nothing is seen, ensure LED is OFF  
    digitalWrite(ledPin, LOW);  
    return;  
  }  
  // Check if any learned object (like your water bottle) is in view  
  if (huskylens.available()) {  
    HUSKYLENSResult result = huskylens.read();  
    // Check if the detected object is "Object 1"
```

```
if (result.command == COMMAND_RETURN_BLOCK && result.ID == 1) {  
  Serial.println("Water Bottle (Object 1) detected!");  
  digitalWrite(ledPin, HIGH); // Turn LED ON  
}  
else {  
  // If a different object is seen, turn LED OFF  
  digitalWrite(ledPin, LOW);  
}  
}  
else {  
  // If the camera sees nothing at all, turn LED OFF  
  digitalWrite(ledPin, LOW);  
}  
}
```

Task 2

```
#include "HUSKYLENS.h"  
  
#include "SoftwareSerial.h"  
  
HUSKYLENS huskylens;  
  
SoftwareSerial mySerial(4, 5); // RX, TX  
  
const int redPin = 9;  
  
const int greenPin = 10;
```

```
const int bluePin = 11;

void setup() {

  Serial.begin(115200);

  mySerial.begin(9600);

  pinMode(redPin, OUTPUT);

  pinMode(greenPin, OUTPUT);

  pinMode(bluePin, OUTPUT);

  clearLED();

  while (!huskylens.begin(mySerial)) {

    Serial.println(F("Begin failed!"));

    delay(500);

  }

  Serial.println("HuskyLens Ready for 5 Colors!");

}

void loop() {

  if (!huskylens.request()) {

    clearLED();

    return;

  }

  if (huskylens.available()) {

    HUSKYLENSResult result = huskylens.read();

    if (result.command == COMMAND_RETURN_BLOCK) {
```

```
switch (result.ID) {  
  case 1: // RED  
    setColor(255, 0, 0);  
    break;  
  case 2: // BLUE  
    setColor(0, 0, 255);  
    break;  
  case 3: // GREEN  
    setColor(0, 255, 0);  
    break;  
  case 4: // YELLOW (Red + Green)  
    setColor(255, 255, 0);  
    Serial.println("Detected: Yellow");  
    break;  
  case 5: // PURPLE (Red + Blue)  
    setColor(255, 0, 255);  
    Serial.println("Detected: Purple");  
    break;  
  default:  
    clearLED();  
    break;  
}
```

```
}  
  
} else {  
  
clearLED();  
  
}  
  
}  
  
void setColor(int redValue, int greenValue, int blueValue) {  
  
analogWrite(redPin, redValue);  
  
analogWrite(greenPin, greenValue);  
  
analogWrite(bluePin, blueValue);  
  
}  
  
void clearLED() {  
  
setColor(0, 0, 0);  
  
}
```

5.0 Data Collection

COLOR	DETECTED ID	OUTPUT
RED	1	RED ON
BLUE	2	BLUE ON
GREEN	3	GREEN ON
YELLOW	4	YELLOW ON
PURPLE	5	PURPLE ON

6.0 Data Analysis

The results show that the HuskyLens AI Vision Camera can detect and identify the selected colors accurately. Each color tested, which includes red, blue, green, yellow, and purple, was correctly assigned to its corresponding ID from 1 to 5. When a color was detected, the Arduino Uno responded by turning on the matching color on the RGB LED. This shows that the communication between the HuskyLens and the Arduino worked properly and the program logic for controlling the LED was implemented correctly.

The outcome also indicates that the color recognition feature of the HuskyLens is reliable under the given conditions. There were no incorrect detections observed during the experiment, which suggests that the training process for each color was effective. Overall, the system was able to respond quickly and accurately to visual input, demonstrating a successful integration between the AI vision sensor and the microcontroller. This setup can be useful for applications that require simple and reliable color detection such as sorting systems or indicator devices.

7.0 Results

The experiment was carried out to evaluate the performance of the HuskyLens AI Vision Camera in detecting predefined colors and producing the corresponding output through an RGB LED. The system was able to successfully identify all five trained colors, namely red, blue, green, yellow, and purple. Each color was consistently assigned to its respective ID, ranging from 1 to 5, as configured during the training stage. When a color object was presented to the camera, the HuskyLens detected it and transmitted the corresponding ID to the Arduino Uno via serial communication. The Arduino then processed this information and activated the RGB LED to display the detected color in real time. The LED output matched the detected color in every test, indicating that both the detection and response processes were functioning correctly. Overall, the system operated as expected without any noticeable delay or incorrect detection during the experiment. The results confirm that the integration between the HuskyLens camera and the Arduino Uno is effective for real time color detection and output indication.

8.0 Discussion

The results of this experiment show that the HuskyLens AI Vision Camera is capable of accurately detecting and classifying different colors when properly trained. The system was able to consistently recognize each predefined color and assign the correct ID, which was then used by the Arduino Uno to control the RGB LED output. This indicates that the overall system integration, including the camera, microcontroller, and LED module, worked smoothly. The real time response of the LED also shows that the communication between the HuskyLens and Arduino through serial communication is efficient and reliable.

One important observation from this experiment is the advantage of using an AI based vision sensor compared to traditional color sensors. The HuskyLens simplifies the process of color detection by handling the image processing internally, reducing the need for complex programming or external software. This makes the system more user friendly and flexible, especially for applications where multiple objects or varying shapes are involved. However, the performance of the system can still be influenced by environmental factors such as lighting conditions, shadows, and background colors. Proper training and consistent lighting are important to ensure accurate detection.

In addition, this experiment highlights the importance of proper system calibration and training. The accuracy of the detection depends heavily on how well the colors are trained into the HuskyLens. If the training is not done carefully, the system may produce incorrect results or fail to detect certain colors. Despite these limitations, the system demonstrates strong potential for practical applications such as automated sorting systems, traffic signal control, and interactive devices. Overall, the experiment provides a clear understanding of how AI vision systems can be integrated with embedded systems to create efficient and responsive mechatronic solutions.

9.0 Conclusion

In conclusion, this experiment successfully demonstrated the use of the HuskyLens AI Vision Camera for color detection and its integration with an Arduino Uno for real time output control. The system was able to accurately identify all the trained colors and produce the correct response through the RGB LED. This confirms that the communication between the vision sensor and the microcontroller is reliable, and the overall system functions as intended. The experiment also highlights the effectiveness of AI based vision sensing compared to traditional methods, as it simplifies the process of color recognition without requiring complex programming or external image processing tools. At the same time, it shows that factors such as proper training and stable lighting conditions are important to maintain accuracy and consistency. Overall, this lab provides valuable hands on experience in combining vision sensors with embedded systems. It strengthens the understanding of sensor integration, serial communication, and real time system response, while also demonstrating how such systems can be applied in practical engineering applications.

10.0 Recommendations

For future improvement, the system can be enhanced to perform more reliably under varying lighting conditions. While the HuskyLens provides accurate color detection in a controlled environment, changes in brightness, shadows, or ambient light may affect its performance. Implementing a consistent lighting setup, such as adding a dedicated LED light source or enclosing the system, can help reduce these variations and improve detection accuracy. This enhancement would make the system more stable and better suited for real-world applications where environmental conditions are less predictable.

11.0 References

1. HuskyLens Repository

<https://github.com/HuskyLens/HUSKYLENSArduino/>

12. Acknowledgements

Special thanks to Dr. ZULKIFLI BIN ZAINAL ABIDIN and Dr. WAHJU SEDIONO for their guidance and support during this experiment.

Student's Declaration

Certificate of Originality and Authenticity

We hereby certify that we are collectively responsible for the work presented in this report. The content reflects our original work, except where specific references or acknowledgements have been made. No part of this report has been completed or submitted by individuals or sources not identified within.

Furthermore, we confirm that this report is the result of a collaborative effort, with contributions made by all group members. The extent of each individual's contribution is documented within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Name: AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI	Read	[/]
Matric Number: 2310663	Understand	[/]
Signature: <i>SYAHMI</i>	Agree	[/]

Name: ALYA SYAFINA BINTI ABDUL RAZAK	Read	[/]
Matric Number: 2218084	Understand	[/]
Signature: <i>SYAF</i>	Agree	[/]